

# From Synthetic Benchmarks to Controlled Solana Testbeds

Author Name

Affiliation

`email@example.org`

Working draft v1.0 empirical pilot

## Abstract

Synthetic benchmarks have historically provided simplified and repeatable workloads for comparing computing systems, but contemporary distributed systems require a broader evaluation model. They are telemetry-rich, resource-constrained, dynamically operated, and increasingly shaped by feedback mechanisms. This paper argues that synthetic benchmarking is therefore evolving from open-loop performance measurement towards reproducible experimental testbeds in which workload generation, observability, dataset production, and future control interfaces are designed together. The argument is developed as a methodological contribution and illustrated through a containerised private Solana network consisting of a validator, wallet and workload components, Yellowstone/Geyser-based observability, monitoring services, and Prometheus-compatible metrics. The paper also reports a preliminary single-host empirical pilot consisting of three baseline runs, two low transaction-load runs, and two medium transaction-load runs. The pilot is not presented as a general Solana performance study. Instead, it validates the article’s measurement discipline: the data-collection pipeline preserves hostname-aware meta-data, validator health samples, RPC-level slot and transaction counters, and workload-side transaction evidence. The results show stable health across all included runs and a clear increase in transaction-count deltas under controlled workload profiles. The resulting case study positions the Solana testbed as a reproducible research infrastructure for studying high-performance blockchain behaviour under controlled synthetic workloads and for preparing later feedback-control experiments with rule-based controllers, classical baselines, model-predictive approaches, and multi-agent reinforcement learning.

**Keywords:** synthetic benchmark; distributed systems; Solana; testbed; observability; Geyser; Yellowstone gRPC; Prometheus; feedback control; multi-agent reinforcement learning; reproducible research.

## 1 Introduction

Performance evaluation has long been a central activity in computer systems research. Synthetic benchmarks made it possible to compare machines, compilers, operating systems, databases, and networked services using workloads that were deliberately simplified, repeatable, and portable. Classical examples such as Whetstone and Dhrystone were designed to capture representative computational behaviour using compact programs rather than complete applications. Later benchmark families expanded the idea towards application suites, database transaction processing, decision support workloads, graph analytics, machine learning, and energy-aware evaluation [10, 13].

This historical trajectory is often described as a progression from simple synthetic programs towards richer, domain-specific suites. That description is accurate but incomplete. The most important transition in modern distributed systems is not merely that benchmarks have become

larger or more realistic. Rather, the benchmark is increasingly becoming an experimental environment. It contains a workload generator, an observability layer, a configuration interface, a dataset pipeline, and, in advanced cases, a feedback controller. This change reflects the operational reality of distributed systems: throughput and latency remain important, but they no longer exhaust the research problem. A modern benchmark must also help answer how a system behaves under perturbation, how it exposes internal state, how reproducible its measurements are, and whether measured behaviour can be used to make safe adaptive decisions.

High-performance blockchain networks provide a useful case for this shift. They combine distributed consensus, transaction processing, resource contention, event streams, RPC interfaces, and strong requirements for reproducibility. Solana is a particularly relevant example because it is designed for high-throughput operation and exposes validator-side streaming mechanisms such as Geyser plugins. The Agave validator documentation describes Geyser as a plugin mechanism through which validator information about accounts, slots, blocks, and transactions can be transmitted to external systems, partly addressing the risk that validators under heavy RPC loads fall behind the network [1]. Yellowstone gRPC builds on this Geyser interface by providing a gRPC-based streaming path for slots, blocks, transactions, and account updates [12].

This paper uses a containerised private Solana local network as a concrete case study. The associated public repository describes a containerised local Solana testbed for dataset generation and latency/performance research, and frames it as a migration from a virtual-machine-based local testbed towards a reproducible Podman/Docker Compose architecture [8]. The repository also references persistent Zenodo records for a Solana v1.18.25 local testbed and for a Yellowstone/Geyser observability extension [7, 9].

The central thesis is as follows:

Synthetic benchmarking in distributed systems is evolving from the measurement of isolated performance limits towards the construction of reproducible, observable, and eventually controllable experimental environments.

Accordingly, the Solana example is not treated as a narrow engineering demonstration. It is used to articulate a more general methodological pattern: a benchmark becomes scientifically stronger when it is designed as a closed research loop connecting workload, observation, artefact production, and control.

## 1.1 Research questions

The article is organised around four research questions. They are deliberately methodological rather than benchmark-result questions, because the current paper is concerned with the design of an experimental infrastructure and the evidence required for future empirical claims.

Table 1: Research questions guiding the article

| ID  | Research question                                                                                                                                           | Expected contribution                                                                                                                     |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| RQ1 | How can synthetic benchmarking in distributed systems be reframed from maximum-performance measurement towards observable and reproducible experimentation? | A conceptual model that links workload generation, orchestration, observability, datasets, and control interfaces.                        |
| RQ2 | What design requirements must a benchmark satisfy before it can support feedback-oriented or adaptive control experiments?                                  | A requirement set covering reproducibility, workload parametrisation, observability, metadata, perturbations, and baseline comparability. |
| RQ3 | How can a containerised Solana localnet serve as a concrete case study for a control-oriented distributed-systems testbed?                                  | An architecture mapping Solana validator, Yellowstone/Geyser, monitor services, Prometheus metrics, and datasets to research roles.       |
| RQ4 | What evidence is still required before stronger claims about Solana performance, observability overhead, or learning-based control can be made?             | An implementation-boundary and evidence-plan framework that separates current artefacts from planned empirical validation.                |

## 1.2 Contributions

The paper makes seven contributions.

1. It reframes synthetic benchmarking as a progression from performance measurement to reproducible, observable, and control-ready experimental infrastructure.
2. It clarifies the distinction between a synthetic benchmark, workload generator, testbed, observability layer, dataset, and controller.
3. It positions the argument against related work on benchmark evolution, workload-driven systems, self-management, observability, and learning-based resource allocation.
4. It maps the proposed concepts onto a containerised Solana localnet architecture with a validator, Yellowstone/Geyser observability, monitoring services, Prometheus-compatible metrics, and dataset outputs.
5. It separates design-supported claims from data-dependent claims through an explicit implementation boundary, evidence plan, and empirical data-readiness framework.
6. It specifies an experimental matrix, metric taxonomy, and metadata templates that prepare the manuscript for empirical extension.
7. It adds a preliminary single-host empirical pilot showing that baseline, low-load, and medium-load profiles are distinguishable through RPC-level and workload-side measurements.

## 1.3 Paper structure

Section 2 positions the work against prior benchmarking, workload-driven, control-theoretic, and observability literature. Section 3 defines the main terms. Section 4 reviews the evolution of synthetic benchmarks. Section 5 explains the transition from open-loop to closed-loop benchmarking. Section 6 proposes requirements for control-oriented benchmarks. Section 7 presents the Solana case study and its implementation boundary. Section 9 specifies the proposed experimental design and

matrix. Section 10 reports the preliminary empirical pilot. Section 11 describes an experimental protocol, metric taxonomy, and evidence plan. Section 13 states when additional empirical data become necessary. Section 14 discusses publication of research artefacts. Section 17 defines staged future work. Section 19 discusses threats to validity, and Section 20 concludes.

## 2 Related work and positioning

### 2.1 Benchmark evolution

The starting point for this paper is the long-standing role of synthetic benchmarks in systems research. Classical synthetic programs such as Dhrystone made it possible to compare computing platforms with a compact and repeatable workload, but they also exposed the central limitation of synthetic benchmarking: the more portable and compact the benchmark becomes, the more careful the evaluator must be about representativeness [13]. Survey work on benchmarking history shows that this problem has not disappeared; instead, benchmark design has repeatedly moved towards richer workload suites, domain-specific protocols, and broader quality dimensions such as energy, reproducibility, and application relevance [10].

The present article follows that trajectory but adds a methodological emphasis. It treats the benchmark not only as a workload and a score, but as a research environment that can generate structured evidence and support future control experiments. This shifts attention from ranking systems to constructing experimental infrastructure.

### 2.2 Dynamic workload and feedback-aware benchmarking

Workload-driven database systems provide a useful precedent because they explicitly combine workload observation with adaptive behaviour. The DSB benchmark was proposed for evaluating both traditional and workload-driven database systems, recognising that systems which observe workload feedback require richer evaluation dimensions than static one-shot throughput tests [3, 11]. The lesson is directly relevant to distributed systems beyond databases: once the system or operator can react to observed workload, the benchmark must capture time, state, configuration, and feedback effects.

This paper generalises that point to a Solana-based blockchain testbed. A workload generator alone is insufficient. The testbed must also record how the system was configured, which host produced the data, what telemetry path was active, and whether a future controller changed workload or system parameters during the run.

### 2.3 Self-managing systems and feedback control

The control-oriented framing is related to work on self-managing and autonomic systems. Control-theoretic approaches to self-management emphasise sensors, effectors, reference objectives, feedback loops, and stability analysis [2, 6]. This literature is important because it provides a disciplined alternative to treating adaptation as an ad hoc script or heuristic.

For the present article, the immediate implication is conservative: a testbed should not jump directly from manual benchmarking to learning-based control. It should first establish open-loop baselines, then deterministic workload profiles, then rule-based or classical baselines. Only after these stages are reproducible does it become scientifically meaningful to evaluate more sophisticated controllers.

## 2.4 Observability in high-performance blockchain systems

Observability is a first-class requirement in high-performance blockchain experimentation. External RPC observations can reveal user-facing behaviour, but validator-side event streams can reveal a different view of internal progress. The Agave validator documentation describes Geyser as a plugin mechanism for transmitting information about accounts, slots, blocks, and transactions to external systems [1]. Yellowstone gRPC provides a Geyser-based gRPC interface for streaming Solana slots, blocks, transactions, and account updates [12].

In the proposed testbed, Geyser and Yellowstone are therefore not merely implementation details. They are part of the observability layer that allows benchmark runs to become structured datasets and, later, potential control inputs.

## 2.5 Learning-based and multi-agent resource allocation

Reinforcement learning and multi-agent reinforcement learning are relevant because distributed systems often contain multiple interacting decision points. Recent surveys discuss MARL for resource allocation and distributed decision-making, but also emphasise challenges such as non-stationarity, reward design, coordination, safety, and evaluation complexity [4, 5]. These challenges are particularly important for blockchain testbeds, where a policy that improves one metric may degrade latency, stability, or reproducibility.

Accordingly, this paper treats MARL as a future research direction rather than a current result. The current article focuses on the infrastructure and evidence discipline required before such claims would be credible.

## 2.6 Positioning of this article

The article is positioned between a benchmark survey, a testbed engineering report, and a future control-systems study. It does not claim to provide a final Solana performance ranking. It instead proposes a repeatable way to design, document, and publish a Solana-based experimental environment so that later performance, overhead, and control claims can be supported by traceable artefacts and datasets.

# 3 Terminology and scope

A recurring source of ambiguity in performance research is that the terms benchmark, test, workload, dataset, and testbed are often used interchangeably. This paper uses them in a narrower sense, as summarised in table 2. The distinction matters because a benchmark designed only to rank systems is different from a testbed designed to support repeated experiments, dataset generation, and eventual control.

Table 2: Terminology used in this paper

| Term                | Meaning in this paper                                                                                   | Typical research role                                   |
|---------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Synthetic benchmark | A repeatable artificial workload and measurement protocol.                                              | Compare behaviour under controlled assumptions.         |
| Workload generator  | A component that creates transactions, RPC requests, events, or resource pressure.                      | Control experimental input.                             |
| Testbed             | A reproducible environment containing system components, configuration, orchestration, and measurement. | Run experiments and generate datasets.                  |
| Observability layer | Metrics, logs, traces, event streams, and dashboards that expose internal and external behaviour.       | Convert system behaviour into analysable evidence.      |
| Dataset             | A structured record of measurements, metadata, configuration, and execution context.                    | Support analysis, replication, and model training.      |
| Controller          | A mechanism that changes workload or system configuration using observed state.                         | Support adaptive operation and closed-loop experiments. |

The scope of the paper is intentionally methodological. It does not claim that a single-machine Solana localnet is equivalent to a production blockchain network. Instead, it argues that such a localnet is a useful, reproducible, and inspectable environment for developing experimental methodology before scaling to multi-host and multi-validator configurations.

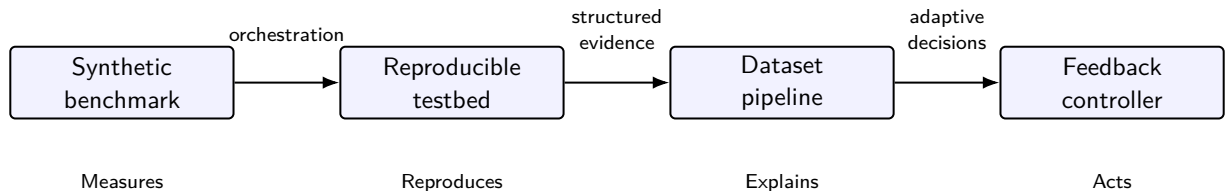


Figure 1: Conceptual progression from benchmark to controlled experimental environment.

## 4 Evolution of synthetic benchmarks

Synthetic benchmarks were originally attractive because they made comparison possible. A compact program or workload could be run across multiple systems, producing a small set of metrics. This simplicity was scientifically useful, but it also created limitations: a synthetic workload may not represent real applications, a single headline score may hide bottlenecks, and optimisations may target the benchmark rather than the underlying system behaviour.

The historical evolution can be viewed as a sequence of broad phases rather than a strict chronology. Early synthetic kernels focused on CPU-level behaviour. Application and suite benchmarks expanded the unit of evaluation. Database and transaction-processing benchmarks introduced domain-specific workloads and more sophisticated measurement protocols. Modern data-intensive and machine-learning benchmarks made the workload multidimensional. More

recent benchmark discussions increasingly include energy, sustainability, reproducibility, dynamic workloads, and feedback-aware systems [3, 10].

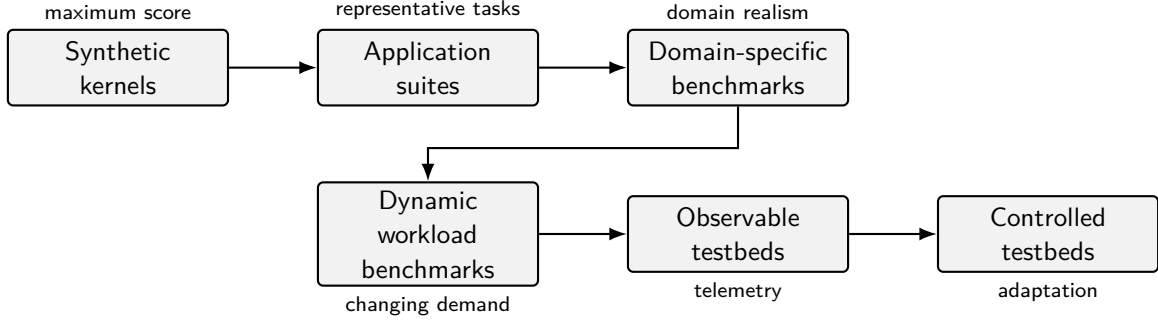


Figure 2: A simplified evolution of synthetic benchmarking concerns.

Table 3: Changing benchmark objectives in computing systems research

| Phase              | Primary question                                            | Typical metric style                                       | Main limitation                            |
|--------------------|-------------------------------------------------------------|------------------------------------------------------------|--------------------------------------------|
| Synthetic kernel   | How fast is a machine on a simplified operation mix?        | Single throughput score.                                   | Weak application realism.                  |
| Application suite  | How does a system perform on a family of workloads?         | Multiple workload scores.                                  | Expensive setup and possible opacity.      |
| Domain benchmark   | How does a system perform in a specific application domain? | Domain-specific throughput, latency, cost, or quality.     | Difficult cross-domain comparison.         |
| Dynamic benchmark  | How does the system behave under changing workloads?        | Time-series performance and adaptation cost.               | Harder reproducibility and interpretation. |
| Observable testbed | What internal and external signals explain behaviour?       | Metrics, logs, events, traces, datasets.                   | Telemetry overhead and data complexity.    |
| Controlled testbed | Can observed behaviour guide safe intervention?             | Stability, responsiveness, safety, objective satisfaction. | Controller design and validation risk.     |

The shift towards workload-driven systems is visible in database benchmarking. The DSB benchmark, for example, was proposed for evaluating both workload-driven and traditional database systems, and its authors emphasise that such systems require additional evaluation dimensions because they use workload observations and feedback to tune behaviour [3, 11]. This is relevant beyond databases: distributed systems increasingly include mechanisms that observe workload and change configuration, placement, resource allocation, batching, admission control, or routing policy.

## 5 From open-loop measurement to closed-loop experimentation

A traditional benchmark is usually open-loop. The evaluator configures a workload, runs the system, collects results, and then analyses them after the fact. This is still essential: without open-loop

baselines, later adaptive experiments cannot be interpreted. However, open-loop benchmarking is insufficient for research questions about self-management, automatic tuning, or resilience.

A closed-loop testbed adds a feedback path. Measurements are not merely stored; they become input to a decision process. The decision process may be manual, scripted, rule-based, control-theoretic, or learning-based. In the language of autonomic computing, sensors obtain measurement data and effectors change resource behaviour; control theory has been proposed as a foundation for building and analysing self-managing systems [2, 6]. The same pattern can be adapted for distributed systems benchmarking.

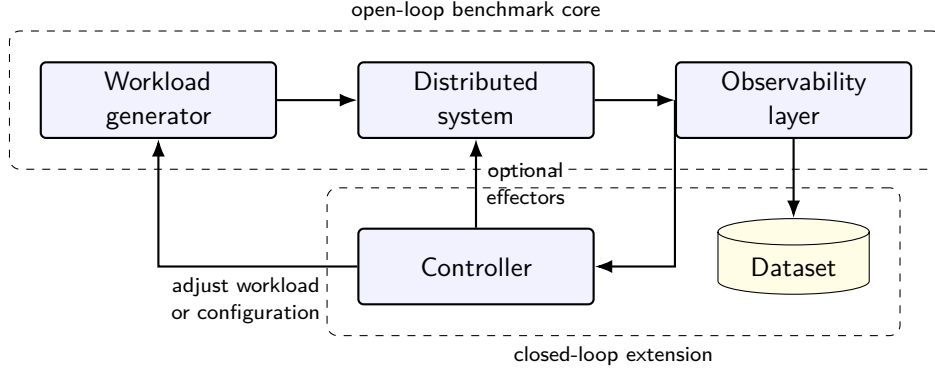


Figure 3: Open-loop benchmarking extended into closed-loop experimentation.

The controller in fig. 3 should be interpreted broadly. The first implementation may simply change a transaction rate according to a scripted schedule. A second implementation may use threshold rules. A third may implement a classical feedback controller or a model-predictive controller. Later implementations may use reinforcement learning or multi-agent reinforcement learning, especially when control actions are distributed across multiple agents and objectives. Recent surveys describe MARL as a framework for distributed decision-making in dynamic resource allocation problems, while also identifying challenges such as non-stationarity, reward design, coordination, and evaluation complexity [4, 5].

This paper therefore treats MARL as a roadmap direction rather than a current claim. A scientifically sound progression should first build reliable measurement, then reproducible datasets, then simple baselines, and only then learning-based controllers.

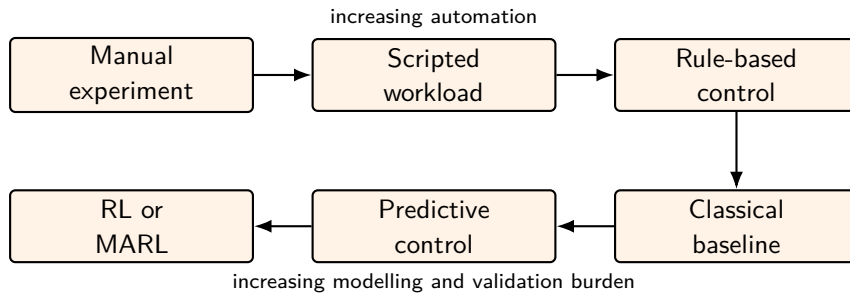


Figure 4: Recommended maturity path for control-oriented benchmarking.

## 6 Requirements for control-oriented distributed benchmarks

A benchmark intended to support future control must be designed differently from a benchmark intended only to produce a maximum throughput number. The workload, observability layer, metadata, and repeatability mechanisms must be part of the benchmark design rather than



afterthoughts.

Table 4: Requirements for control-oriented benchmarking

| Requirement                | Rationale                                                          | Implementation implication                                                                 |
|----------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Reproducible orchestration | Experiments must be rerunnable across hosts and time.              | Use container images, compose files, pinned versions, and recorded image digests.          |
| Workload parametrisation   | Control requires explicit inputs.                                  | Expose transaction rate, request mix, burst profile, duration, and seed.                   |
| Observability coverage     | The system state must be visible enough for diagnosis and control. | Export validator, RPC, Geyser, monitor, container, and host metrics.                       |
| Time alignment             | Distributed metrics must be analysable together.                   | Record timestamps consistently and include clock metadata.                                 |
| Dataset metadata           | Raw measurements are insufficient without execution context.       | Store hostname, commit hash, image tag, image digest, CPU, memory, OS, and Solana version. |
| Perturbation scenarios     | Controllers must be evaluated under changing conditions.           | Include CPU limits, memory pressure, RPC pressure, and workload bursts.                    |
| Baseline comparability     | Adaptive mechanisms must be compared against simple alternatives.  | Run open-loop, static, rule-based, and classical baselines before RL/MARL.                 |
| Publication readiness      | Scientific claims should be inspectable.                           | Publish code, configuration, datasets, figures, and DOI-backed releases.                   |

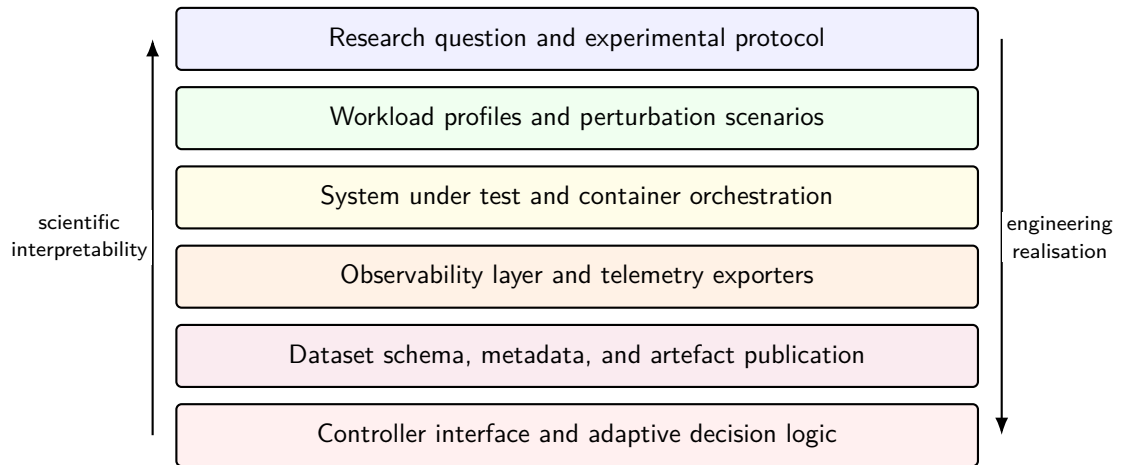


Figure 5: Layered design of a benchmark that can evolve into a control-oriented testbed.

The hostname requirement is particularly important for the Solana project because datasets are intended to be committed or published for later comparison. File names should therefore include at least the experiment identifier, timestamp, hostname, workload profile, and relevant version tag. This makes it possible to distinguish datasets created on different machines even when the workload parameters are identical.

## 7 Solana containerised testbed case study

### 7.1 Rationale for Solana

Solana is an appropriate case study for control-oriented benchmarking because its local execution environment can be studied under controlled workloads while still exposing concepts that matter in production-like distributed systems: transaction submission, validator processing, RPC responsiveness, slot progression, event streaming, and resource contention. The purpose is not to reproduce the public mainnet. The purpose is to create an inspectable private environment that supports reproducible experimentation.

### 7.2 Architecture overview

The case-study architecture consists of a validator, wallet or payer initialisation, a workload generator, Yellowstone/Geyser observability, monitor services, Prometheus-compatible metrics, and a dataset export path. The repository currently frames the project as a containerised local Solana testbed for dataset generation and latency/performance research [8].

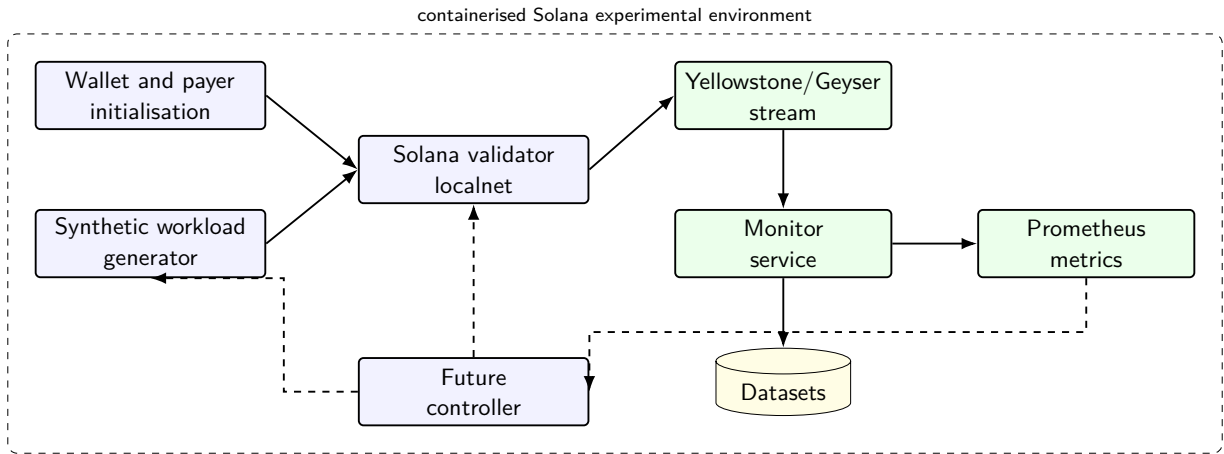


Figure 6: High-level architecture of the Solana case-study testbed. Dashed arrows indicate the planned feedback-control extension.

### 7.3 Component responsibilities

Table 5 maps the major components to their research roles. The design principle is to keep each component observable and replaceable. For example, a simple shell-based workload generator may later be replaced with a Python generator, a Rust generator, or a controller-driven generator without changing the validator container itself.

Table 5: Component responsibilities in the Solana testbed

| Component             | Responsibility                                                                         | Research value                                        |
|-----------------------|----------------------------------------------------------------------------------------|-------------------------------------------------------|
| Validator container   | Runs the private Solana localnet and exposes RPC and validator behaviour.              | System under test.                                    |
| Wallet initialisation | Creates or funds accounts required by experiments.                                     | Reproducible experiment preparation.                  |
| Workload generator    | Submits controlled transaction or RPC pressure.                                        | Defines experimental input.                           |
| Yellowstone/Geyser    | Streams validator-side events such as slots, transactions, blocks, or account updates. | Low-latency observability path.                       |
| Monitor container     | Collects, transforms, and exports selected metrics.                                    | Converts events into datasets and monitoring signals. |
| Prometheus endpoint   | Exposes scrapeable time-series metrics.                                                | Enables standard monitoring and later control input.  |
| Dataset output        | Stores raw and derived metrics with metadata.                                          | Supports replication, analysis, and publication.      |
| Controller interface  | Adjusts workload or configuration based on observed state.                             | Planned closed-loop extension.                        |

#### 7.4 Observability through Geyser and Yellowstone

The Geyser interface is central because it allows validator-side state changes to be exported to external systems. The Agave documentation explains that the validator supports a plugin mechanism for transmitting information about accounts, slots, blocks, and transactions to external stores such as relational databases, NoSQL databases, or Kafka [1]. The Yellowstone gRPC repository describes a Geyser-based gRPC interface for Solana with plugin and sample clients, providing streams for slots, blocks, transactions, and account update notifications over a standardised path [12]. In the proposed testbed, this streaming layer is not merely a data export convenience; it is part of the benchmark design.

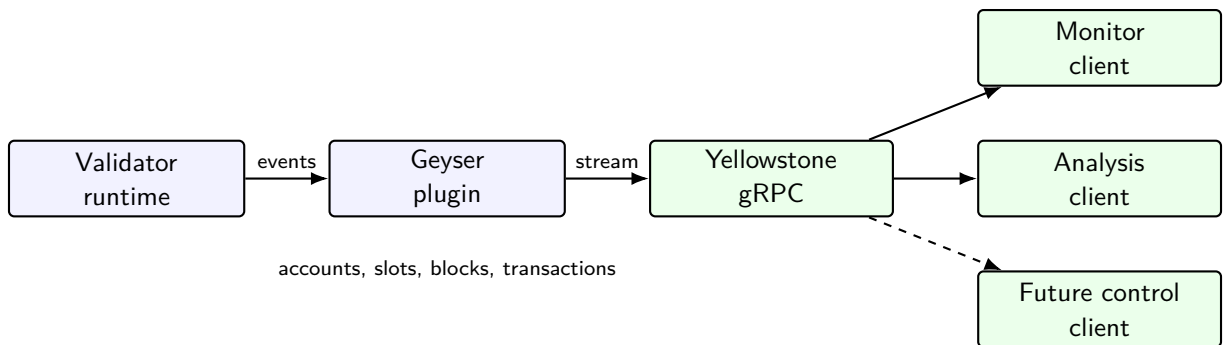


Figure 7: Role of Geyser and Yellowstone gRPC in the observability path.

## 8 Implementation boundary and claim status

A central risk in testbed papers is overclaiming. A repository, a localnet, and an observability extension do not automatically justify claims about production performance, mainnet behaviour,

or the effectiveness of adaptive controllers. This section therefore separates the current article’s methodological claims from claims that require future experimental evidence.

Table 6: Implementation boundary for the current article version

| Area                                   | Status in this article                                                                               | Interpretation for claims                                                                                           |
|----------------------------------------|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Conceptual model                       | Included as the main theoretical contribution.                                                       | Claims may be made about methodology and design rationale.                                                          |
| Containerised Solana localnet          | Treated as the case-study foundation and linked to public artefacts.                                 | Claims should be limited to reproducible local experimental infrastructure unless measured data are added.          |
| Yellowstone/Geyser observability       | Treated as an observability-path extension and research direction supported by referenced artefacts. | Claims may describe architecture and intended observability role; quantitative overhead claims require experiments. |
| Prometheus-compatible metrics          | Included as part of the target monitoring architecture.                                              | Claims should focus on metric taxonomy and dataset design until real scrape data are included.                      |
| Synthetic workload generator           | Included as a required component of the benchmark design.                                            | Workload realism, transaction success, and latency claims require defined workload profiles and recorded runs.      |
| Dataset publication                    | Included as a publication model and metadata requirement.                                            | Dataset claims require concrete files, checksums, host metadata, and release identifiers.                           |
| Feedback controller                    | Explicitly treated as future work.                                                                   | No claim should be made that control, optimisation, RL, or MARL has already been validated.                         |
| Kubernetes and multi-validator scaling | Explicitly treated as future work.                                                                   | The current paper should not generalise single-host results to cluster-level behaviour.                             |

The practical consequence is that the present manuscript should be read as a methodological, artefact-oriented, and experiment-design paper. Its strongest current claims concern framing, requirements, architecture, reproducibility discipline, and the experimental roadmap. Strong empirical claims should be added only after the corresponding datasets and validation scripts are available.

## 9 Experimental design

The purpose of the experimental design is to make the future empirical version of the article executable rather than merely aspirational. Each experiment should identify the independent variable being manipulated, the dependent variables being measured, the artefacts required for replication, and the claim that the experiment is allowed to support. This discipline is especially important for a local blockchain testbed, because small differences in host hardware, container image, ledger reset policy, or observability configuration can change the interpretation of the results.

The design assumes an incremental path. Baseline boot experiments should be completed first, because later workload and observability claims are not meaningful if the localnet cannot be started reproducibly. Transaction-load and RPC-pressure experiments should then exercise the system under controlled synthetic demand. Observability-overhead and Geyser stream-latency experiments should quantify the cost and value of the telemetry path. Resource-constrained and cross-host experiments should test robustness and reproducibility. Finally, control-readiness

experiments should prepare the environment for rule-based or classical feedback baselines before any learning-based controller is introduced.

## 9.1 Experimental matrix

Table 7 defines the proposed experimental matrix. The matrix is not a results table. It is a design table: it states what must be run and recorded before the article can make stronger empirical claims.

Table 7: Experimental matrix for future empirical versions

| Experiment                 | Independent variable                                                        | Dependent variables                                                                            | Required artefact                                                                  |
|----------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Baseline boot              | Container image version, compose file, ledger reset policy.                 | RPC health, slot progression, startup time, container exit status.                             | Boot log, health-check transcript, baseline metrics CSV, metadata JSON.            |
| Transaction load           | Target transaction rate, burst profile, duration, account set, random seed. | Submission count, confirmation count, failure rate, confirmation latency, CPU and memory use.  | Workload log, transaction summary CSV, time-series metrics, workload profile file. |
| RPC pressure               | RPC request rate, request mix, concurrency, duration.                       | RPC latency, error rate, validator health, resource pressure.                                  | RPC pressure log, response-latency CSV, validator-health timeline.                 |
| Observability overhead     | Geyser and monitor enabled or disabled; Prometheus scrape interval.         | CPU delta, memory delta, latency delta, throughput delta, event-processing delay.              | Paired-run metrics, overhead comparison table, run-pair metadata.                  |
| Geyser stream latency      | Event type, monitor client configuration, timestamp source.                 | Validator event count, monitor receive time, stream delay, missing events.                     | Event trace CSV, timestamp alignment notes, stream integrity report.               |
| Resource-constrained run   | CPU quota, memory limit, disk mode, network constraint where available.     | Throughput, latency, errors, restart count, recovery time.                                     | Constraint configuration, container stats, failure/recovery timeline.              |
| Cross-host reproducibility | Hostname, CPU, memory, OS, kernel, container engine.                        | Metric variance across hosts, reproducibility of pass/fail status, distributional differences. | Host metadata JSON files, cross-host comparison CSV, reproducibility summary.      |
| Control-readiness run      | Scripted workload adjustment rule or configuration effector.                | Response time, stability, safety violations, recovery behaviour.                               | Controller transcript, before/after metrics, safety-event log.                     |

## 9.2 Run profiles and sequencing

The experiments should be executed through named run profiles. A profile should be a small, version-controlled description of the workload, duration, observability settings, and expected outputs. A recommended sequence is:

1. **boot-baseline**: start the validator and confirm health without synthetic load.
2. **txload-low**: run a conservative transaction-load profile to validate instrumentation.
3. **txload-step**: increase the target rate in stages to observe latency and resource response.
4. **rpc-pressure**: apply read/query pressure separately from transaction submission.
5. **obs-off** and **obs-on**: run paired profiles for observability-overhead estimation.
6. **geyser-latency**: measure event propagation through the Geyser/Yellowstone path.
7. **resource-limit**: repeat a known workload under constrained CPU or memory.
8. **cross-host**: repeat selected profiles on at least two hosts using identical code and image digests.

For each profile, the article should preserve a distinction between *input parameters*, *measured signals*, and *derived summaries*. Input parameters belong in version-controlled profile files. Measured signals belong in raw datasets. Derived summaries and figures should be reproducible from scripts.

## 9.3 Claim-to-experiment mapping

The experimental design should constrain the strength of future claims. For example, a baseline boot run can support a claim that the localnet starts reproducibly under specified conditions. It cannot support a claim about transaction latency. A paired observability-overhead run can support a claim about the overhead of a specific monitoring configuration on a specific host. It cannot support a general claim about all Geyser deployments. A cross-host run can support a limited reproducibility claim only if host metadata, image digests, and workload profiles are published alongside the metrics.

This claim-to-experiment mapping is deliberately conservative. Its purpose is to make the paper easier to defend in review and easier to extend when real datasets become available.

## 10 Preliminary empirical pilot

The preceding sections define the experimental design and claim discipline required for empirical work. This section adds a preliminary pilot dataset to validate the measurement pipeline. The pilot is intentionally modest: it is a single-host, single-validator localnet study conducted on the host recorded as **fedora\_**, with all included runs using the same Git short commit **d0c848a**. The purpose is not to establish general Solana performance limits. The purpose is to test whether the proposed dataset structure can distinguish baseline localnet operation from controlled transaction-load profiles while preserving run metadata, validator health samples, RPC counters, and workload-side transaction evidence.

The pilot includes seven valid runs: three baseline runs, two low transaction-load runs, and two medium transaction-load runs. A previously collected **txload-low** archive without independent workload evidence is excluded from this pilot and treated as a diagnostic run rather than as transaction-load evidence.

Table 8: Pilot dataset composition

| Profile       | Runs | Workload evidence                                                                                           | Permitted interpretation                                                                     |
|---------------|------|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Baseline      | 3    | No external transaction generator.                                                                          | Validator health and localnet progression under the default containerised configuration.     |
| txload-low    | 2    | Sequential SOL transfers generated from inside the validator container at target 1 transaction per second.  | Low synthetic transaction pressure and workload-side success evidence.                       |
| txload-medium | 2    | Sequential SOL transfers generated from inside the validator container at target 2 transactions per second. | Higher synthetic transaction pressure, constrained by sequential CLI confirmation behaviour. |

Table 9 reports the run-level summary. Each run has 60 RPC samples collected at five-second intervals. All included samples returned healthy validator status. The transaction-load profiles also include a workload-side transaction log, with attempted, successful, and failed transaction counts.

Table 9: Run-level empirical pilot summary

| Run timestamp    | Profile  | Health | Slot $\Delta$ | Tx count $\Delta$ | Workload ok | Failed | Mean ms |
|------------------|----------|--------|---------------|-------------------|-------------|--------|---------|
| 2026-05-15 11:15 | baseline | 60/60  | 759           | 759               | —           | —      | —       |
| 2026-05-15 11:37 | baseline | 60/60  | 758           | 758               | —           | —      | —       |
| 2026-05-15 11:47 | baseline | 60/60  | 768           | 768               | —           | —      | —       |
| 2026-05-15 12:46 | low      | 60/60  | 734           | 1000              | 265         | 0      | 791.3   |
| 2026-05-15 13:25 | low      | 60/60  | 733           | 998               | 264         | 0      | 798.1   |
| 2026-05-15 13:14 | medium   | 60/60  | 741           | 1102              | 360         | 0      | 763.2   |
| 2026-05-15 13:32 | medium   | 60/60  | 739           | 1102              | 362         | 0      | 760.3   |

Table 10: Profile-level empirical pilot summary

| Profile       | Runs | Health  | Mean slot $\Delta$ | Mean tx $\Delta$ | Mean workload ok | Achieved tx/s | Mean p95 ms |
|---------------|------|---------|--------------------|------------------|------------------|---------------|-------------|
| baseline      | 3    | 180/180 | 761.7              | 761.7            | —                | —             | —           |
| txload-low    | 2    | 120/120 | 733.5              | 999.0            | 264.5            | 0.94          | 883.5       |
| txload-medium | 2    | 120/120 | 740.0              | 1102.0           | 361.0            | 1.29          | 840.5       |

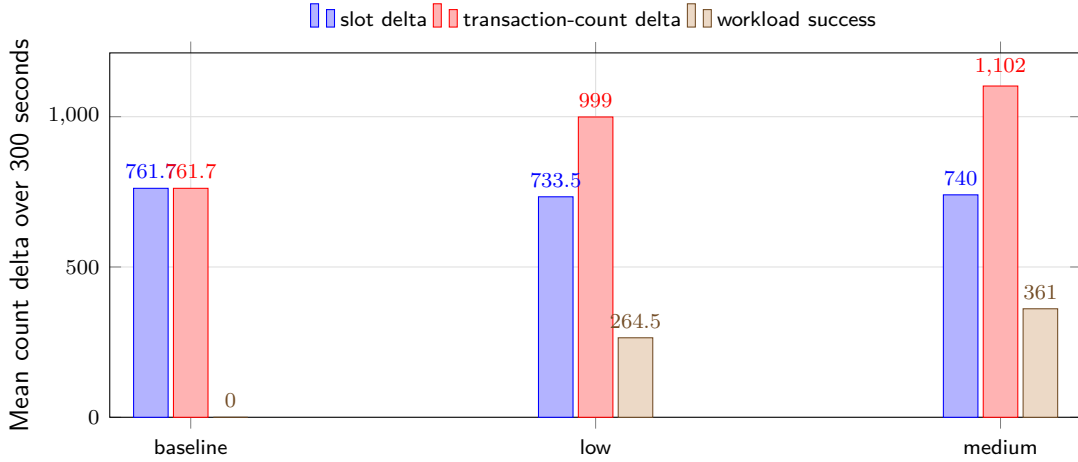


Figure 8: Mean slot progression, transaction-count progression, and workload-side successes in the pilot dataset. The figure is intended to validate measurement separability rather than to rank Solana performance.

The main finding is methodological. In the baseline runs, the transaction-count delta is approximately equal to the slot delta. In the transaction-load profiles, the transaction-count delta is greater than the slot delta by an amount that is consistent with the workload-side successful transfer counts. For the low profile, the mean transaction-count delta is 999.0 while the mean workload-side success count is 264.5. For the medium profile, the corresponding values are 1102.0 and 361.0. This alignment indicates that the collection pipeline can connect an externally named workload profile, validator RPC counters, and workload-side transaction evidence.

The pilot also exposes an important engineering limitation. The workload generator used sequential `solana transfer` commands and waited for transaction confirmation. Consequently, the achieved rates were lower than the nominal target rates: approximately 0.94 transactions per second for the low profile and 1.29 transactions per second for the medium profile. This is acceptable for the present pilot because the goal is not saturation. It does, however, show that future higher-pressure experiments should use an asynchronous or batched workload generator rather than sequential CLI transfers.

### 10.1 Pilot claim boundary

The pilot supports four limited claims. First, the localnet remained healthy throughout the included 300-second runs. Second, the dataset bundle format is sufficient to preserve host metadata, Git identity, compose files, RPC metrics, and workload evidence. Third, low and medium synthetic transaction-load profiles are distinguishable from baseline behaviour in transaction-count deltas. Fourth, the workload-side success counts are consistent with the additional RPC-level transaction-count progression.

The pilot does not support claims about public Solana mainnet performance, multi-validator behaviour, Geyser stream latency, observability overhead, or controller performance. Those claims require the additional experiments described in the experimental matrix. The next empirical extension should therefore add paired observability-on/off runs and a higher-quality workload generator before making stronger performance claims.



## 11 Experimental protocol and metric taxonomy

A useful experimental protocol must separate the scientific question, workload profile, system configuration, metric collection, and dataset publication. The protocol should also record enough context to make the experiment reproducible on another host.

### 11.1 Experiment classes

Table 11 defines a minimal set of experiment classes for the next version of the testbed paper. These experiments are intentionally staged. The first group validates reproducibility. The second group measures behaviour under synthetic workload. The third group evaluates the cost and value of observability. The fourth group prepares the system for feedback control.

Table 11: Proposed experiment classes

| Experiment class        | Purpose                                                          | Main signals                                          | Expected output                        |
|-------------------------|------------------------------------------------------------------|-------------------------------------------------------|----------------------------------------|
| Baseline boot           | Validate reproducible localnet startup.                          | Health status, slot progression, RPC availability.    | Pass/fail log and baseline metrics.    |
| Transaction load        | Evaluate controlled synthetic transaction pressure.              | Submission rate, success rate, latency, resource use. | Time-series dataset and summary table. |
| RPC pressure            | Study validator responsiveness under read/query load.            | RPC latency, error rate, validator health.            | Responsiveness profile.                |
| Observability overhead  | Compare runs with and without monitoring/Geyser.                 | CPU, memory, latency, event delay.                    | Overhead estimate.                     |
| Geyser stream latency   | Estimate event propagation delay through the observability path. | Event timestamp differences, queueing behaviour.      | Stream latency dataset.                |
| Resource constraint     | Evaluate behaviour under CPU or memory limits.                   | Throughput, latency, failures, recovery time.         | Perturbation profile.                  |
| Dataset reproducibility | Repeat selected runs across hosts.                               | Metric distributions, host metadata, variance.        | Cross-host comparison table.           |
| Control readiness       | Test scripted or rule-based adjustment.                          | Response to workload changes and safety violations.   | Baseline for future controllers.       |

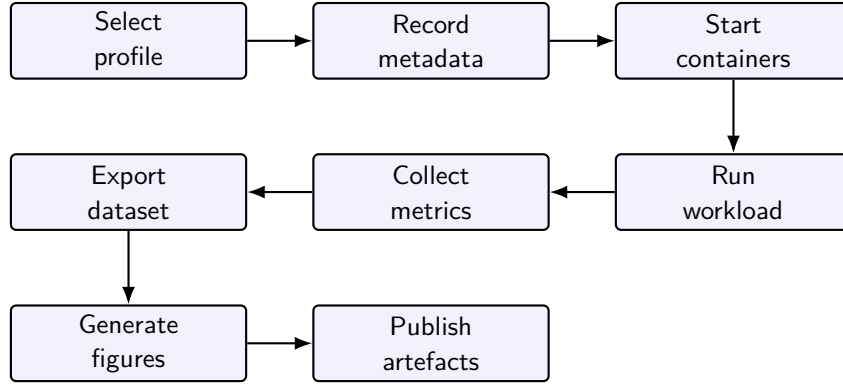


Figure 9: Recommended experiment lifecycle for dataset-producing benchmark runs.

## 11.2 Metric taxonomy

The testbed should avoid reducing performance to one metric. Throughput and latency remain important, but they must be interpreted together with validator health, observability delay, RPC responsiveness, and resource consumption.

Table 12: Metric taxonomy for the Solana testbed

| Metric group        | Examples                                                                      | Interpretation                                      |
|---------------------|-------------------------------------------------------------------------------|-----------------------------------------------------|
| Workload input      | Target rate, burst size, request mix, duration, random seed.                  | What the experiment asked the system to do.         |
| Transaction outcome | Submitted, confirmed, failed, retried, dropped.                               | Whether the workload succeeded.                     |
| Latency             | Submission latency, confirmation latency, RPC response latency, stream delay. | User-visible and observability-path responsiveness. |
| Validator state     | Health status, slot progression, block or transaction event counts.           | Internal progress of the localnet.                  |
| Geyser stream       | Event count, stream gaps, client receive time, monitor processing time.       | Observability-path behaviour.                       |
| Container resources | CPU, memory, network I/O, disk I/O, restart count.                            | Runtime resource pressure.                          |
| Host metadata       | Hostname, CPU model, memory, OS, kernel, container engine.                    | Reproducibility and cross-host analysis.            |
| Build metadata      | Git commit, image tag, image digest, Solana version, compose file.            | Artefact traceability.                              |

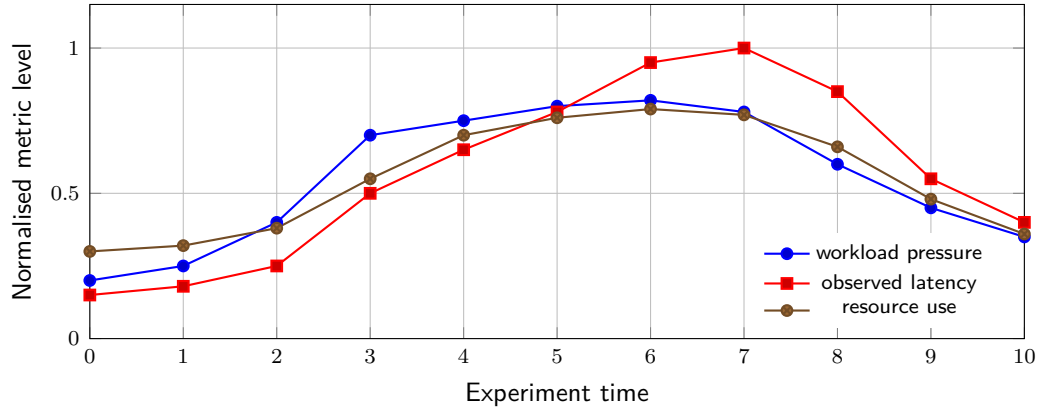


Figure 10: Illustrative time-series view for multi-metric interpretation. The values are schematic placeholders rather than measured results.

### 11.3 Dataset naming and metadata

Dataset files should be self-describing enough to remain useful outside the original machine. A recommended naming pattern is:

```
YYYYMMDDTHHMMSSZ_hostname_profile_duration_gitshortsha_metrics.csv
```

For example:

```
20260515T121500Z_hpz440_txload-low_300s_a1b2c3d_metrics.csv
```

The file name should not replace a metadata sidecar file. Each dataset should also include a structured metadata record such as JSON or YAML.

Table 13: Recommended dataset metadata fields

| Field group            | Fields                                                      | Reason                          |
|------------------------|-------------------------------------------------------------|---------------------------------|
| Run identity           | run id, timestamp, hostname, operator, project version.     | Distinguish repeated runs.      |
| Source identity        | Git repository, commit hash, branch, dirty flag.            | Link dataset to code state.     |
| Image identity         | Container image tags, image digests, compose file.          | Recreate runtime environment.   |
| Host identity          | CPU model, memory, OS, kernel, container engine version.    | Explain cross-host differences. |
| Workload identity      | Profile name, rate, duration, seed, request mix.            | Reproduce input pressure.       |
| Solana identity        | Solana/Agave version, validator flags, ledger mode.         | Reproduce system under test.    |
| Observability identity | Geyser config, monitor version, Prometheus scrape interval. | Interpret telemetry path.       |
| Output identity        | Raw files, derived files, figure files, checksum.           | Validate publication artefacts. |

## 12 Evidence plan for empirical versions

The current article now includes a preliminary empirical pilot, but it remains conservative about generalisation. A later results-oriented version should connect every stronger substantive claim to a dataset, metric, and reproducibility record. Table 14 defines a practical evidence plan.

Table 14: Evidence plan for converting methodological claims into empirical claims

| Claim type                   | Required evidence                                                                           | Primary metric or artefact                                                        | Current status                               |
|------------------------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|----------------------------------------------|
| Reproducible startup         | Repeated clean launches on at least one host, with recorded versions and logs.              | Validator health, slot progression, RPC availability, compose file, image digest. | Pilot evidence included.                     |
| Workload handling            | Runs with defined synthetic transaction or RPC profiles.                                    | Submission rate, success rate, confirmation latency, error count.                 | Pilot evidence included.                     |
| Observability path function  | Evidence that Yellowstone/Geyser events are received and transformed by monitor components. | Event counts, stream timestamps, monitor receive times, missing-event checks.     | Architecture defined.                        |
| Observability overhead       | Paired runs with and without observability components.                                      | CPU, memory, latency, throughput, stream delay.                                   | Future experiment.                           |
| Cross-host reproducibility   | Repeated profiles on two or more machines.                                                  | Hostname-aware metric distributions and metadata sidecars.                        | Future experiment.                           |
| Dataset traceability         | Published raw data, derived data, scripts, figures, and checksums.                          | DOI records, Git commit hash, checksums, README mapping.                          | Pilot bundle validated; publication pending. |
| Rule-based control readiness | Scripted adjustment of workload or configuration using observed state.                      | Response time, safety violations, stability, recovery behaviour.                  | Future work.                                 |
| RL or MARL suitability       | Baselines, environment definition, reward design, and safety constraints.                   | Baseline comparison, training logs, evaluation seeds, policy robustness.          | Long-term roadmap.                           |

This evidence plan also protects the article from premature generalisation. It makes clear which claims are already supported by design artefacts and which require future data collection.

## 13 Data requirements for future empirical extension

The current manuscript includes a small empirical pilot, but additional empirical data become necessary when the manuscript is revised to make stronger measured claims about performance, observability overhead, Geyser/Yellowstone latency, cross-host reproducibility, or controller behaviour. This distinction is important because it separates the validity of the proposed research infrastructure and pilot pipeline from the validity of future numerical results.

Table 15 defines the practical trigger points for requesting datasets. It should be read as an editorial control mechanism: if a claim remains architectural or methodological, the current artefacts, protocols, and pilot dataset are sufficient; if a claim becomes broader or more quantitative, the corresponding run bundle must be collected and archived.

Table 15: When empirical data become required

| Manuscript claim                               | Data requirement                                                           | Minimum acceptable input                                                                          |
|------------------------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Methodological claim about benchmark evolution | No new dataset required.                                                   | Literature-supported argument and conceptual diagrams.                                            |
| Architectural claim about the Solana testbed   | No performance dataset required, but repository state must match the text. | Current compose files, README, release notes, and DOI records.                                    |
| Claim that the localnet starts reproducibly    | Baseline boot data required.                                               | At least three clean boot runs with health checks, slot progression, logs, and metadata.          |
| Claim about transaction-load behaviour         | Transaction-load dataset required.                                         | Low and medium load profiles with latency, success rate, errors, CPU, memory, and run metadata.   |
| Claim about observability overhead             | Paired comparison required.                                                | Matched runs with observability disabled and enabled, using the same workload and host.           |
| Claim about Geyser or Yellowstone latency      | Stream-timing dataset required.                                            | Event timestamps, monitor receive timestamps, stream counts, and missing-event checks.            |
| Claim about cross-host reproducibility         | Multi-host dataset required.                                               | The same profile executed on at least two hostnames with complete host metadata.                  |
| Claim about feedback control                   | Controller-run dataset required.                                           | Baseline open-loop run, rule-based run, safety conditions, controller logs, and recovery metrics. |

For a first empirical revision, the most efficient dataset package would contain three baseline runs and two transaction-load profiles on one host. A stronger revision would add paired observability-overhead runs and one repeated profile on a second host. Learning-based control data should not be collected until open-loop, scripted, and rule-based baselines are stable.

## 14 Research artefacts and publication model

Reproducible benchmarking should publish not only the article but also the code, configuration, datasets, and derived figures. The Solana project already has a public GitHub repository and DOI-backed Zenodo records referenced from the repository README [8]. This creates a foundation for a stronger research artefact model.

Table 16: Current and planned artefact classes

| Artefact class         | Current role                                           | Recommended next step                                                     |
|------------------------|--------------------------------------------------------|---------------------------------------------------------------------------|
| GitHub repository      | Source code, compose files, scripts, documentation.    | Ensure README status, release notes, and article claims are synchronised. |
| Zenodo software record | Persistent reference for versioned software state.     | Use DOI in article and preserve exact release archive.                    |
| Zenodo technical note  | Persistent explanation of design and validation state. | Link technical note to article methodology.                               |
| Raw datasets           | Evidence from individual benchmark runs.               | Include hostname and metadata sidecars.                                   |
| Derived datasets       | Cleaned or aggregated results.                         | Publish processing scripts with checksums.                                |
| Figures                | Visual evidence used in article.                       | Generate from scripts rather than manual editing.                         |
| Experiment protocol    | Exact run procedure.                                   | Store as markdown and LaTeX appendix.                                     |

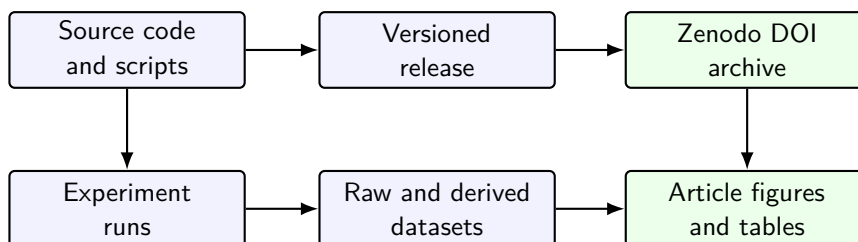


Figure 11: Recommended publication pipeline linking repository, releases, datasets, and article evidence.

Before submission, the repository should be checked for consistency between the implemented state and the article text. Any section describing planned work should be clearly separated from completed work. This is especially important where observability, monitoring, Docker Hub images, and DOI releases are mentioned, because readers may compare article claims directly with the repository.

## 15 Repository integration strategy

The article should be integrated into the public repository as a maintained research artefact rather than as a detached manuscript. Repository integration has two purposes. First, it makes the article source, bibliography, build instructions, and generated PDF traceable to a specific project state. Second, it reduces the risk of divergence between the manuscript, README files, release notes, datasets, and Zenodo records.

The recommended integration model is conservative: keep the LaTeX source in a dedicated article directory, keep dataset templates and empirical datasets outside the article source tree, and link the article from the research-program documentation. The article should not become the only source of truth about implementation status. Instead, it should cite and summarise repository state that is also documented in README and release files.

## 15.1 Recommended repository layout

A practical layout is shown in table 17. The exact directory names may be adjusted to match the repository, but the separation of roles should be preserved: article source, project documentation, dataset specifications, empirical data, and publication records should remain distinguishable.

Table 17: Recommended repository integration layout

| Path                                                  | Role                                                                                 | Policy                                                                             |
|-------------------------------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <code>paper/synthetic-benchmark-s-solana/</code>      | LaTeX source, bibliography, build instructions, and generated PDF for this article.  | Commit source files and build notes. Commit PDF when useful for review or release. |
| <code>docs/research-program/</code>                   | Narrative project documentation and links to the article.                            | Keep short status pages here; avoid duplicating the full article.                  |
| <code>docs/research-program/article-status.md</code>  | Human-readable status of article versions, claims, and next actions.                 | Update on each article milestone.                                                  |
| <code>data/raw/</code>                                | Future raw experimental outputs from validator, workload, monitor, and metrics runs. | Commit only small or curated data; use Zenodo for larger datasets.                 |
| <code>data/derived/</code>                            | Cleaned or aggregated datasets generated from raw data.                              | Regenerate from scripts where possible.                                            |
| <code>results/figures/</code>                         | Generated figures from empirical data.                                               | Store generated plots with script references.                                      |
| <code>scripts/</code>                                 | Build, validation, dataset collection, and plotting scripts.                         | Prefer deterministic scripts over manual figure editing.                           |
| <code>prompts/codex/</code>                           | Optional local Codex prompts for repository maintenance tasks.                       | Use for repeatable integration or documentation updates.                           |
| <code>releases/</code> or <code>docs/releases/</code> | Release notes and DOI mapping.                                                       | Keep GitHub and Zenodo identifiers synchronised.                                   |

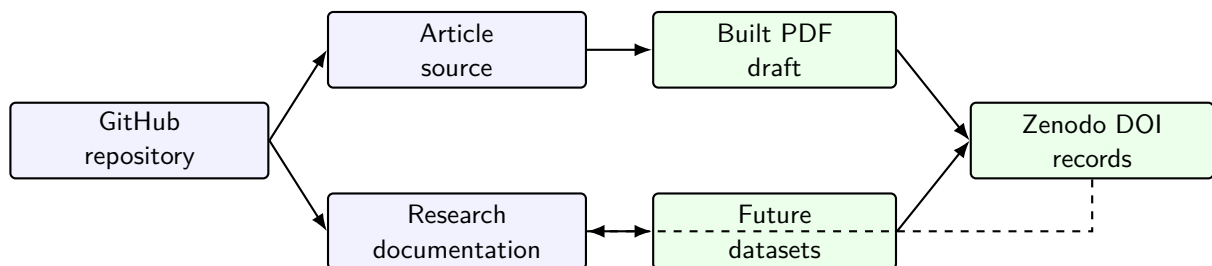


Figure 12: Repository integration map for article source, documentation, generated PDF, future datasets, and DOI records.

## 15.2 Integration rules

The following rules should be used when the article is committed to the repository.

1. Keep the manuscript source in a dedicated article directory and avoid scattering LaTeX files across the repository.
2. Treat the generated PDF as a review artefact. It may be committed for convenience, but the source files and build instructions are authoritative.
3. Keep raw empirical data outside the article directory. The article should reference datasets; it should not become a data dump.
4. Maintain a short status page that states the article version, current claim boundary, completed artefacts, missing empirical evidence, and next planned action.
5. Update Zenodo and GitHub release references only when the repository state has actually been released or archived.
6. When empirical datasets are added later, include metadata, host information, workload profiles, logs, and checksums before adding quantitative claims to the article.

## 15.3 Suggested integration commit

A suitable first repository integration commit should be narrow and descriptive. It should add the article source, repository-facing README files, and status documentation, without claiming that empirical datasets are already available. A recommended commit message is:

```
docs: add synthetic benchmarking article integration workflow
```

If the generated PDF is also committed, the commit body should say that the PDF is a derived review artefact generated from the LaTeX source. This prevents confusion about whether the PDF or the source is the maintained object.

## 16 Operational GitHub integration workflow

The repository integration step should be performed as a small documentation-only change. Its purpose is to place the article and its supporting files into the repository, confirm that the manuscript can be rebuilt locally, inspect the staged changes, and push a narrow commit. It should not modify validator, wallet, monitor, container, or workload-generation implementation code.

The workflow in table 18 assumes a local clone of the repository and an extracted article integration package. It deliberately separates copying files, building the PDF, inspecting the working tree, committing, and pushing. This separation makes the integration auditable and reversible.



Table 18: Operational GitHub integration workflow

| Stage       | Action                                                                                                | Expected result                                                                                                                                         |
|-------------|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pre-flight  | Confirm the repository branch, clean working tree, and location of the extracted integration package. | The user knows which branch will receive the article files.                                                                                             |
| Copy        | Copy article source, PDF, build files, documentation, prompts, and scripts into their target paths.   | New or updated files appear under <code>paper/</code> , <code>docs/research-program/</code> , <code>prompts/codex/</code> , and <code>scripts/</code> . |
| Build       | Run the article build script from the repository root.                                                | <code>paper/synthetic-benchmarks-solana/main.pdf</code> is regenerated successfully.                                                                    |
| Inspect     | Review <code>git status --short</code> and, when useful, <code>git diff --stat</code> .               | Only article and documentation files are changed.                                                                                                       |
| Stage files | Stage the article, documentation, prompt, and helper scripts explicitly.                              | No logs, temporary files, raw datasets, or implementation changes are staged by accident.                                                               |
| Commit      | Commit with a documentation-oriented message.                                                         | The repository contains a traceable article integration commit.                                                                                         |
| Push        | Push the branch to GitHub and verify it online.                                                       | GitHub shows the article source, review PDF, and status documentation.                                                                                  |

A conservative command sequence is shown below. The paths should be adjusted to the local machine, but the ordering should be preserved.

```
# 1. Enter the local repository.
cd ~/project/solana-containerised-testbed

# 2. Confirm branch and cleanliness.
git branch --show-current
git status --short

# 3. Define the extracted integration package location.
PKG=~/.Downloads/solana_benchmark_article_v1_0/repository-integration

# 4. Copy the article and repository-facing files.
bash "$PKG/scripts/integrate_synthetic_benchmark_article_v1_0.sh" \
  "$PKG" \
  "$(pwd)"

# 5. Rebuild the PDF from the repository copy.
bash scripts/build_synthetic_benchmarks_article.sh \
  paper/synthetic-benchmarks-solana

# 6. Inspect the working tree.
git status --short
git diff --stat
```

```
# 7. Stage only article and documentation files.
git add \
  paper/synthetic-benchmarks-solana \
  docs/research-program/synthetic-benchmarks-solana-article.md \
  docs/research-program/article-status.md \
  prompts/codex/21_github_integration_workflow_v1_0.md \
  scripts/build_synthetic_benchmarks_article.sh \
  scripts/integrate_synthetic_benchmark_article_v1_0.sh \
  scripts/check_synthetic_benchmark_article_integration.sh

# 8. Commit and push.
git commit -m "docs: add synthetic benchmarking article integration workflow"
git push
```

The helper integration script is intentionally limited. It copies files into documentation and article paths, makes helper scripts executable, and prints the resulting status. It does not commit, push, delete datasets, or change runtime implementation files. The final decision about staging and committing remains with the repository maintainer.

## 16.1 Rollback and safety checks

If the integration was copied into the wrong branch or wrong repository, the change should be reverted before staging or committing. A safe rollback before committing is:

```
git restore --staged \
  paper/synthetic-benchmarks-solana \
  docs/research-program \
  prompts/codex \
  scripts/build_synthetic_benchmarks_article.sh \
  scripts/integrate_synthetic_benchmark_article_v1_0.sh \
  scripts/check_synthetic_benchmark_article_integration.sh

git restore \
  paper/synthetic-benchmarks-solana \
  docs/research-program \
  prompts/codex \
  scripts/build_synthetic_benchmarks_article.sh \
  scripts/integrate_synthetic_benchmark_article_v1_0.sh \
  scripts/check_synthetic_benchmark_article_integration.sh
```

For newly added untracked files, inspect them first and then remove only the intended paths. Avoid broad cleanup commands unless the working tree has been reviewed carefully.

```
git status --short
rm -rf paper/synthetic-benchmarks-solana
rm -f docs/research-program/synthetic-benchmarks-solana-article.md
rm -f docs/research-program/article-status.md
rm -f prompts/codex/21_github_integration_workflow_v1_0.md
rm -f scripts/integrate_synthetic_benchmark_article_v1_0.sh
rm -f scripts/check_synthetic_benchmark_article_integration.sh
```

## 17 Roadmap towards adaptive and multi-agent control

The proposed roadmap intentionally moves from measurement to control in conservative stages. This reduces the risk of presenting a complex learning-based controller before the underlying measurement pipeline is stable.

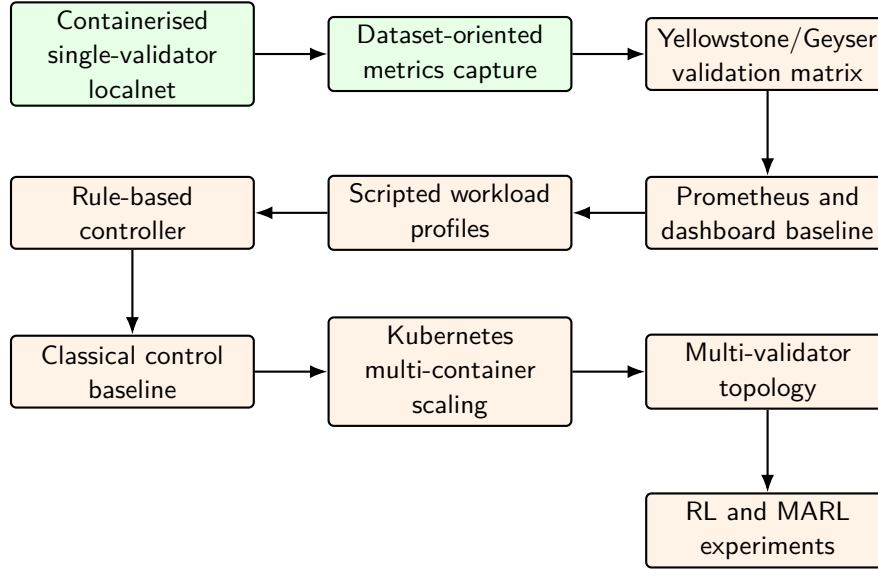


Figure 13: Suggested roadmap from the present Solana localnet to adaptive and multi-agent experiments. Status labels should be verified before article submission.

### 17.1 Why not start directly with MARL

Multi-agent reinforcement learning is attractive because distributed systems naturally involve multiple interacting components and partially local decisions. However, MARL introduces evaluation risks. Rewards may encode the wrong behaviour. Agents may overfit to a narrow synthetic environment. Training may produce unstable policies. Comparative baselines may be weak. For these reasons, MARL should appear in the article as a long-term research direction supported by the testbed, not as an immediate result.

A staged approach is stronger:

1. Build reproducible open-loop experiments.
2. Publish datasets with complete metadata.
3. Implement deterministic workload profiles.
4. Add rule-based controllers and simple safety constraints.
5. Compare against classical feedback or model-predictive baselines.
6. Only then evaluate RL or MARL policies.

## 18 Discussion

The main methodological benefit of the proposed approach is that it treats benchmarking as a research infrastructure problem. A headline throughput result may be useful, but it is not enough to support reproducible science. A control-oriented testbed must expose what was run, where it was run, how it was measured, and how the measurements can be used for later decision-making.

The Solana case study also shows why observability is not a secondary concern. In a high-performance blockchain environment, the difference between external RPC observations and

validator-side event streams can be scientifically meaningful. Geyser and Yellowstone gRPC provide a natural path for studying this difference, because they expose validator-side event flow to external consumers [1, 12]. The monitor component can then transform these events into Prometheus-compatible metrics and dataset records.

At the same time, the approach has limits. A single-host localnet cannot reproduce public-network dynamics such as adversarial network delay, heterogeneous validators, stake distribution, mainnet transaction composition, or validator-client diversity. It should therefore be interpreted as a controlled laboratory environment. Its value is strongest in early-stage methodology development, instrumentation testing, workload design, and reproducibility studies.

## 19 Threats to validity

Table 19: Threats to validity and mitigation strategies

| Threat                  | Risk                                                                          | Mitigation                                                                          |
|-------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Localnet realism        | A private single-validator network may not represent public Solana behaviour. | Frame claims as testbed methodology, not mainnet performance.                       |
| Synthetic workload bias | Artificial transaction mixes may miss real application patterns.              | Define multiple workload profiles and document assumptions.                         |
| Host dependence         | Results may depend strongly on CPU, memory, disk, and container engine.       | Include hostname and hardware metadata in every dataset.                            |
| Observability overhead  | Geyser, monitor, and metrics exporters may affect the system being measured.  | Run with and without observability components where possible.                       |
| Clock alignment         | Event timing may be distorted by clock differences or timestamp placement.    | Record timestamp sources and use consistent time standards.                         |
| Version drift           | Solana, Agave, Yellowstone, and container images may change over time.        | Pin versions and record image digests.                                              |
| Controller overclaiming | Future RL or MARL claims may exceed what the testbed validates.               | Use staged baselines and clearly distinguish implemented work from roadmap.         |
| Publication mismatch    | Repository state may diverge from article claims.                             | Synchronise README, release notes, DOI records, and article text before submission. |

## 20 Conclusion

This paper argued that synthetic benchmarking in distributed systems is evolving from a tool for measuring performance towards a methodology for building reproducible, observable, and eventually controllable experimental environments. The proposed framing does not discard traditional performance metrics. Instead, it places them inside a wider research loop that includes workload generation, telemetry, dataset publication, and control interfaces.

The containerised Solana localnet case study illustrates this transition. A validator-centred

private network, augmented with Yellowstone/Geyser observability, monitor services, Prometheus-compatible metrics, and DOI-backed research artefacts, can serve as a practical foundation for studying high-performance blockchain behaviour under controlled conditions. The preliminary empirical pilot strengthens the paper by showing that the data-collection workflow can distinguish baseline execution from controlled transaction-load profiles while preserving metadata, RPC counters, and workload-side transaction evidence. All included pilot samples returned healthy validator status, and transaction-load profiles produced transaction-count deltas consistent with the workload-side successful transfer counts.

The pilot should not be read as a final performance study. It is a single-host validation of the measurement pipeline and article methodology. The next repository step is to integrate the article and derived pilot summaries through a narrow GitHub commit, using the operational workflow specified in this manuscript. The next empirical step is to extend the matrix with paired observability-overhead runs, Geyser/Yellowstone stream-latency measurements, and cross-host repetitions. Only after these baselines are stable should the work move towards rule-based control, control-theoretic baselines, and later multi-agent reinforcement learning experiments.

## A Proposed figure list for the final article

Table 20: Planned visual material for the article

| No. | Figure                          | Purpose                                                                               |
|-----|---------------------------------|---------------------------------------------------------------------------------------|
| 1   | Conceptual progression          | Show transition from benchmark to testbed, dataset, and controller.                   |
| 2   | Benchmark evolution timeline    | Summarise historical development of benchmark goals.                                  |
| 3   | Open-loop versus closed-loop    | Explain the control-oriented framing.                                                 |
| 4   | Control maturity path           | Present staged progression towards MARL.                                              |
| 5   | Requirement layers              | Connect scientific and engineering layers.                                            |
| 6   | Solana testbed architecture     | Show validator, workload, monitor, Geyser, and datasets.                              |
| 7   | Geyser observability path       | Explain validator-side streaming.                                                     |
| 8   | Experiment lifecycle            | Show repeatable benchmark procedure.                                                  |
| 9   | Time-series metric illustration | Demonstrate multi-metric analysis style.                                              |
| 10  | Artefact publication pipeline   | Link code, release, DOI, datasets, and paper evidence.                                |
| 11  | Repository integration map      | Show how article source, documentation, datasets, PDF, and DOI records are connected. |
| 12  | Research roadmap                | Show future development stages.                                                       |

## B Proposed table list for the final article

Table 21: Planned tables for the article

| No. | Table                      | Purpose                                                                                |
|-----|----------------------------|----------------------------------------------------------------------------------------|
| 1   | Research questions         | State the methodological questions guiding the article.                                |
| 2   | Terminology                | Prevent confusion between benchmark, testbed, workload, dataset, and controller.       |
| 3   | Benchmark objectives       | Compare historical phases and limitations.                                             |
| 4   | Requirements               | Define design requirements for control-oriented benchmarking.                          |
| 5   | Implementation boundary    | Separate completed artefacts, target architecture, and future work.                    |
| 6   | Component responsibilities | Map Solana components to research roles.                                               |
| 7   | Experimental matrix        | Specify variables, measured signals, and required artefacts for future empirical work. |
| 8   | Experiment classes         | Define the empirical validation classes.                                               |
| 9   | Metric taxonomy            | Organise metrics for analysis and control.                                             |
| 10  | Dataset metadata           | Standardise reproducibility fields.                                                    |
| 11  | Evidence plan              | Connect future empirical claims to required datasets and metrics.                      |
| 12  | Data-readiness triggers    | Specify when empirical datasets become necessary.                                      |
| 13  | Artefact classes           | Define publication outputs.                                                            |
| 14  | Repository layout          | Define where the article, documentation, scripts, and future datasets should live.     |
| 15  | Threats to validity        | Support rigorous discussion.                                                           |

## C Repository integration checklist

Before committing the article to the repository, use the following checklist.

- Create a dedicated article directory such as `paper/synthetic-benchmarks-solana/`.
- Copy `main.tex`, `references.bib`, `main.bbl`, `main.pdf`, and `README.md` into the article directory.
- Add a repository-facing status document under `docs/research-program/`.
- State clearly that the current manuscript is methodological and does not yet contain empirical Solana performance results.
- Add build instructions that work with BibTeX and with the included `main.bbl` fallback.
- Do not place raw datasets inside the article directory.
- Commit dataset templates separately from empirical datasets.
- Check that `README`, article, release notes, and Zenodo records use the same terminology for completed work and future work.

A conservative copy plan is:

```
paper/synthetic-benchmarks-solana/
main.tex
references.bib
main.bbl
```

```

main.pdf
README.md
README_BUILD.md

docs/research-program/
  synthetic-benchmarks-solana-article.md
  article-status.md

prompts/codex/
  21_github_integration_workflow_v1_0.md

scripts/
  build_synthetic_benchmarks_article.sh
  integrate_synthetic_benchmark_article_v1_0.sh
  check_synthetic_benchmark_article_integration.sh

```

## D Local GitHub integration command sheet

This appendix provides a compact command sheet for integrating the article into a local repository clone. It is intended for repository maintenance rather than for article review.

```

# Local assumptions:
# - repository clone: ~/project/solana-containerised-testbed
# - extracted v1.0 package: ~/Downloads/solana_benchmark_article_v1_0

cd ~/project/solana-containerised-testbed

git branch --show-current
git status --short

PKG=~/.Downloads/solana_benchmark_article_v1_0/repository-integration

bash "$PKG/scripts/integrate_synthetic_benchmark_article_v1_0.sh" \
  "$PKG" \
  "$(pwd)"

bash scripts/build_synthetic_benchmarks_article.sh \
  paper/synthetic-benchmarks-solana

bash scripts/check_synthetic_benchmark_article_integration.sh

git status --short
git diff --stat

git add \
  paper/synthetic-benchmarks-solana \
  docs/research-program/synthetic-benchmarks-solana-article.md \
  docs/research-program/article-status.md \
  prompts/codex/21_github_integration_workflow_v1_0.md \

```

```
scripts/build_synthetic_benchmarks_article.sh \
scripts/integrate_synthetic_benchmark_article_v1_0.sh \
scripts/check_synthetic_benchmark_article_integration.sh
```

```
git commit -m "docs: add synthetic benchmarking article integration workflow"
git push
```

A slightly more explicit commit body is:

Add article source, generated review PDF, repository-facing status pages, local integration helpers, and a Codex prompt for the synthetic benchmarking article. The draft remains methodological and does not introduce empirical Solana performance, overhead, stream-latency, reproducibility, or controller claims.

## E Data handover checklist for an empirical revision

When the manuscript moves from the present methodological version to an empirical version, the first dataset handover should be small, structured, and easy to audit. A recommended first archive should contain one directory per run and should avoid mixing raw data, derived data, and figures.

Table 22: Recommended first data handover package

| Package element       | Minimum content                                         | Purpose                                                               |
|-----------------------|---------------------------------------------------------|-----------------------------------------------------------------------|
| Baseline runs         | Three clean validator/localnet boot runs.               | Establish reproducible startup and health-check behaviour.            |
| Transaction-load runs | Two profiles, preferably low and medium load.           | Provide the first latency, success-rate, and resource-use evidence.   |
| Observability pair    | One matched workload run with observability off and on. | Estimate the monitoring and Geyser/Yellowstone overhead.              |
| Run metadata          | One <code>metadata.json</code> per run.                 | Link every measurement to version, host, workload, and configuration. |
| Host metadata         | One <code>host.json</code> per host.                    | Explain cross-host and hardware-dependent differences.                |
| Raw metrics           | CSV or JSONL files with timestamps.                     | Preserve primary evidence without manual editing.                     |
| Logs                  | Validator, workload, and monitor logs.                  | Support debugging and interpretation.                                 |
| Checksums             | One <code>checksums.sha256</code> per run bundle.       | Protect against accidental file changes after publication.            |

The preferred upload format is a single ZIP archive. The archive should be named with a project prefix and dataset version, for example `solana-testbed-datasets-v0.1.zip`. If the first archive is large, a smaller pilot archive with one baseline run and one transaction-load run should be reviewed first.



## F Dataset metadata templates

This appendix defines practical metadata templates for future dataset-producing runs. The exact field names may evolve with the repository, but every empirical version of the article should preserve the same principle: a dataset is incomplete unless it records the run identity, host identity, source identity, container identity, workload identity, observability settings, and output checksums.

### F.1 Minimal run metadata JSON

```
{
  "run_id": "20260515T121500Z_hpz440_txload-low_300s",
  "timestamp_utc": "2026-05-15T12:15:00Z",
  "hostname": "hpz440",
  "operator": "okhoshaba",
  "project": "solana-containerised-testbed",
  "git_repository": "https://github.com/okhoshaba/solana-containerised-testbed",
  "git_branch": "main",
  "git_commit": "a1b2c3d",
  "git_dirty": false,
  "compose_file": "compose.yellowstone.yaml",
  "container_engine": "podman",
  "container_engine_version": "REPLACE_WITH_VERSION",
  "solana_version": "1.18.25",
  "agave_or_validator_image": "REPLACE_WITH_IMAGE_TAG",
  "image_digest": "sha256:REPLACE_WITH_DIGEST",
  "workload_profile": "txload-low",
  "duration_seconds": 300,
  "random_seed": 12345,
  "geyser_enabled": true,
  "yellowstone_enabled": true,
  "prometheus_enabled": true,
  "prometheus_scrape_interval_seconds": 5,
  "ledger_reset": true,
  "notes": "Short free-text note about the run."
}
```

### F.2 Host metadata JSON

```
{
  "hostname": "hpz440",
  "os": "Ubuntu 24.04 LTS",
  "kernel": "REPLACE_WITH_KERNEL",
  "cpu_model": "REPLACE_WITH_CPU_MODEL",
  "cpu_cores": 8,
  "memory_gb": 32,
  "disk_type": "SSD",
  "container_engine": "podman",
  "container_engine_version": "REPLACE_WITH_VERSION",
  "clock_sync": "REPLACE_WITH_NTP_OR_CHRONY_STATUS"
}
```

### F.3 Workload profile YAML

```
profile_name: txload-low
profile_version: 1
purpose: Conservative transaction-load validation profile.
duration_seconds: 300
warmup_seconds: 30
cooldown_seconds: 30
target_transaction_rate_per_second: 5
burst_mode: false
request_mix:
  transfer_transactions: 1.0
random_seed: 12345
payer_account: default
recipient_account_policy: generated
expected_outputs:
  - metrics_csv
  - workload_log
  - metadata_json
  - summary_json
```

### F.4 Recommended dataset bundle

A complete run bundle should use a stable directory name based on timestamp, hostname, profile, duration, and short commit hash. A minimal bundle should contain:

```
20260515T121500Z_hpz440_txload-low_300s_a1b2c3d/
  metadata.json
  host.json
  workload-profile.yaml
  container-images.txt
  compose-file-copy.yaml
  raw-metrics.csv
  workload.log
  validator-health.jsonl
  summary.json
  checksums.sha256
```

The file `checksums.sha256` should be generated after the run and before publication. Derived datasets and article figures should not overwrite raw files; they should be stored separately or generated by scripts from immutable raw inputs.

## G Draft checklist before submission

- Replace placeholder author metadata.
- Verify the current repository state against every claim in the paper.
- Verify that the article source is stored in a dedicated repository directory with build instructions.
- Verify that repository-facing documentation links to the article without duplicating the full manuscript.

- Check that README, release notes, Zenodo records, and article claims use the same implementation status.
- Add exact Solana, Agave, Yellowstone, Docker/Podman, and Prometheus versions.
- Record container image tags and image digests for every experiment.
- Add profile files for boot-baseline, txload-low, txload-step, rpc-pressure, obs-off, obs-on, geyser-latency, resource-limit, and cross-host runs.
- Request empirical datasets only when the manuscript introduces quantitative performance, overhead, stream-latency, reproducibility, or controller claims.
- Replace the schematic time-series figure with measured results.
- Add measured observability-overhead results from paired runs.
- Add hostname-aware dataset examples from at least two machines if available.
- Publish raw data, derived data, plotting scripts, metadata JSON files, workload profiles, and checksums.
- Confirm Zenodo DOI metadata and cite exact versions.
- Ensure all future work is clearly separated from completed implementation.
- Review whether any claim implies mainnet behaviour; if so, either add evidence or narrow the claim.

## References

- [1] Anza. Solana validator geyser plugins. <https://docs.anza.xyz/validator/geyser/>, 2026. Accessed 2026-05-15.
- [2] Yixin Diao, Joseph L. Hellerstein, Sujay Parekh, Rean Griffith, Gail Kaiser, and Dan Phung. Self-managing systems: A control theory foundation. In *Proceedings of the IEEE International Conference on Local Computer Networks*, 2005. <https://www.cs.cmu.edu/~15849g/readings/diao05.pdf>.
- [3] Bailu Ding, Surajit Chaudhuri, Johannes Gehrke, and Vivek Narasayya. DSB: A decision support benchmark for workload-driven and traditional database systems. *Proceedings of the VLDB Endowment*, 14(13):3376–3388, 2021. <https://www.vldb.org/pvldb/vol14/p3376-ding.pdf>.
- [4] Mohammadreza Doostmohammadian et al. Survey of distributed algorithms for resource allocation over multi-agent systems. *Annual Reviews in Control*, 2025. [https://people.kth.se/~kallej/papers/Survey\\_ARC2025\\_Doostmohammadian.pdf](https://people.kth.se/~kallej/papers/Survey_ARC2025_Doostmohammadian.pdf).
- [5] Mohamad A. Hady, Siyi Hu, Mahardhika Pratama, Jimmy Cao, and Ryszard Kowalczyk. Multi-agent reinforcement learning for resources allocation optimization: A survey. *Artificial Intelligence Review*, 2025. doi: 10.1007/s10462-025-11340-5.
- [6] Joseph L. Hellerstein. Self-managing systems: A control theory foundation. <https://www.cs.cmu.edu/~15849g/readings/hellerstein05.pdf>, 2004.
- [7] Oleksandr Khoshaba. Containerising a Solana v1.18.25 local testbed for reproducible dataset generation on legacy x86\_64 CPUs. Zenodo, 2026. doi: 10.5281/zenodo.20098291.
- [8] Oleksandr Khoshaba. Solana containerised testbed. <https://github.com/okhoshaba/solana-containerised-testbed>, 2026. Accessed 2026-05-15.

- [9] Oleksandr Khoshaba. Yellowstone/Geyser observability extension for the Solana containerised testbed v0.2.0. Zenodo, 2026. doi: [10.5281/zenodo.20167936](https://doi.org/10.5281/zenodo.20167936).
- [10] MDPI Computers. Relevance and evolution of benchmarking in computer systems: A survey. *Computers*, 2025. <https://www.mdpi.com/2073-431X/14/12/516>.
- [11] Microsoft Research. DSB: A decision support benchmark for workload-driven and traditional database systems. <https://www.microsoft.com/en-us/research/publication/dsb-a-decision-support-benchmark-for-workload-driven-and-traditional-database-systems/>, 2026. Accessed 2026-05-15.
- [12] Triton One / rpcpool. Yellowstone gRPC: A geyser based gRPC interface for Solana. <https://github.com/rpcpool/yellowstone-grpc>, 2026. Accessed 2026-05-15.
- [13] Reinhold P. Weicker. Dhrystone: A synthetic systems programming benchmark. *Communications of the ACM*, 27(10):1013–1030, 1984. doi: [10.1145/358274.358283](https://doi.org/10.1145/358274.358283).